COMP 4501 - Project final deliverable

Deep in the Trenches

Lily Barker, Melanie Thorpe, Nicholas Ralph, Hendry Hu

Setting

The year is 4119. Humanity's interstellar expeditions have bore fruit with the first surveys of Wolf 359 b, an ultramassive terrestrial exoplanet, revealing rare high-energy crystal deposits and compacted bio-organic material. In the scramble to extract these resources, competing corporations have descended on the planet, where space-based non-aggression pacts do not apply. Wolf 359 b's crushing gravitational field and powerful electromagnetic storms render high-tech armaments and combat vehicles inoperable, leaving only bio-mechanical troops and constructions synthesized from the planet's highly adapted organic goo capable of functioning. These unique conditions have reversed millennia of military doctrine, resulting in a situation where mobility is limited, the defensive advantage is significant, skirmishes over resource-rich pockets have devolved into stalemate as troops dig in at the front lines.

As orbital commander, your job is to win control of these pockets by establishing territorial control and breaking the stalemate by whatever means necessary. High-grav bio-mechanical troops and armaments are highly limited upon initial deployment, so you will be required to set up on-site resource extraction and production facilities to scale up your combat capacity, construct elaborate trench networks, and defend your front line against an enemy who is doing the same.

How to Play

Deep in the Trenches has singleplayer and multiplayer modes. For singleplayer, choose "Play with a computer" in the main menu. Alternatively, if you want to try out the multiplayer you can run multiple instances of Godot (Debug > Custom Run Instances > Enable Multiple Instances > 2) and then have one select "Host" and the other "Join", allowing you to join the same lobby as opposing players.

<u>Controls:</u>
Q and **E** allow the player to zoom in and out.
W, **A**, **S,** and **D** allow the player to move the camera around.
SHIFT increases the camera's movement speed.
CTRL toggles the excavation path tool. Holding the **left mouse button**, and dragging the cursor through the terrain, marks a path for excavation. Scrolling the **mouse wheel** changes the depth of the path, allowing the player to create shallower/deeper trenches. Pressing **ENTER** submits the path, summoning available troops to come excavate.

Debug Controls (not actually part of gameplay, just for debugging):
Holding **ALT** and clicking spawns troops.
Holding **SHIFT** and **left mouse button** adds more terrain, while **SHIFT** and **right mouse button** digs into the terrain.

The player's goal is to capture as much of the map as possible, secure and excavate the resources, and then take out the opposing corporation (whether that be the other player, or the computer).

To do so, the player must dig trenches - in order to protect their troops, but also to collect resources.

They can use these resources to create defenses, like turrets or mortars, or they can create factories to produce more soldiers, bullets, magazines and artillery shells. The player can take control of their soldiers, assigning them roles or giving them direct orders, to ensure that all resources are extracted and transported, and that all defenses are manned and fully loaded. They can also set the targets for their mortars, to ensure their front line holds strong - or to give them the offensive edge.

Implementation

Pathfinding

Some relevant files:

- hendry/navigation/navigation.gd (API for navigation)
- hendry/navigation/nav_map.gd (the internal worker class where all the pathfinding and terrain sampling actually happens)
- hendry/navigation/types/ (types that helps with providing the rest of the game with requests and responses from the API, as well as types used internally for threading and queuing jobs)
- hendry/navigation/agent_config.gd (base resource for an agent's self-defined cost function)

The pathfinding requirement is met through implementing a custom navigation API that wraps around Godot's inner AStarGrid2D. Instead of using Godot's ready-made navigation system, the dynamic nature of the terrain meant that we had to build our own grid for A* navigation using samples of the characteristics of the terrain at each point on a grid. Upon a pathfinding request, the navigation data (and thus the cost) is fetched on-the-fly from the terrain. That navigation data is stored in a dictionary that updates when the terrain changes, but only the parts that were changed. This speeds up navigation data access. However, the real challenge comes from the

sheer number of points in the A* grid, slowing down navigation dramatically when there are many requests at once.

To mitigate the slowdowns from navigation, pathfinding runs off of the main thread. Instead, it runs on 8 worker threads. To make the navigation system compatible with multithreading, nav_map.gd is encapsulated and moved onto its own worker thread, while the navigation.gd API handles thread creation and the shared queue. It also handles giving the worker threads all the data it will need upon starting a request, as well as publishing the results from worker threads onto the main thread. One exception to this pattern is accessing the terrain. Since the terrain object is too big to just make a copy for the worker thread every time, each worker has their own persistent terrain object that gets incrementally updated.

Finally, a big feature of our pathfinding system is agent-defined cost functions. Since each agent has different behaviours for navigation, it can attach its own custom pathfinding function that uses a set of available terrain characteristics (as well as its own predefined cost layers). This allows for the most important part of pathfinding in our project, which is to get units to walk inside the trenches instead of taking the shortest path if possible. This sets up the gameplay of trench warfare.

Animation

The animation requirement is met through the foot (soldier) units in the game. Soldier units are imported with a skeleton which turns into a Skeleton3D node when imported to Godot. To make the soldier animations, the Skeleton3D bones are connected to an Animation Player and the animations are controlled by our state machine. The animations are done by transforming the Skeleton3D's bones and then inserting the transformations as key frames into the animation player. Godot then handles the smooth transitions between key frame transformations.

There are 5 different animations included in the project:

- walking
- shooting
- picking up an item
- dropping an item
- idling

For the sake of performance idling is just a static animation. Using the state machine, the animations are played based on the current state of the unit.

1. If a unit is in a waiting position in any of its states the idle animation plays on loop.
2. If a unit is going from point A to point B in any of its states the walking animation plays on loop.

3. When a unit digs the grabbing animation is played to show it picking up the materials off the ground.
4. The grabbing animation is also reversed when the unit is dropping off resources at one of the buildings, or played normally when it is taking resources from one of the buildings.

The grabbing and dropping animations are not played on loop, they are both a one shot animation. During the animations, the weapon is also moved accordingly, remaining on the side of the player except when shooting. The shooting animation is played while the unit is currently attacking or sees an enemy unit and switches to the state automatically.

Semi-automatic actions

This requirement is met through the foot units in the game. Foot units have a hierarchical finite state machine that modifies behaviour based on a set of rules. Types of states include excavate, transport and patrol. When a user is not given a direct action, like attack, the unit will decide their own actions based on these rules. For example, if a dig path has been selected by the player, the unit will switch to the excavate state and go dig along that path. Once that unit's inventory is full of organic material, it will either switch to the idle state or the transport state if an item depot is nearby. Once it empties its inventory it will continue excavating if there is more to dig. Another one of the states, which takes importance above all is attacking, if a foot unit spots an enemy nearby it will immediately switch to attacking it.

Units can also be assigned roles, which will prioritize different states (so that - in the case of a conflict - it knows which state to choose first). For example, if the troop is assigned the role 'Excavator', it will choose to dig whenever possible (unless self defense is required, as that overrides default actions).

Enemy AI

Enemy units function identically to player units. The enemy player AI is implemented in terms of managing their units in the same manner that a player would. As implemented, the enemy player AI marks out a preset pattern of trenches to dig at the beginning of the game, and places a predetermined set of buildings to be constructed. During gameplay, the enemy player AI will manage its units such that all operable stations (turrets, mortars, etc) are manned, and a predetermined ratio of assigned unit roles (25% item transport, 25% excavation, 50% patrol) is maintained.

Game mechanics

Players can command troops to do specific actions (e.g. move or shoot), assign troops a role (e.g. Excavate or Patrol), assign troops to workstations, construct buildings or set trench excavation paths.

Units actions are divided into 2 categories, direct actions and roles. Direct actions include things like moving safely or directly and shooting, these are actions that happen once and then stop. Roles are like priorities: basically, you're telling this unit that if they have a choice between multiple states in their state machine, to prioritize the one that pertains to their role. Various roles can be assigned to units including excavate, transport and patrol. Excavate tells the unit to look for an excavation path and dig it out, transport tells the unit to bring resources from their inventory or an item depot to a building under construction. Patrol tells the unit to walk around the area safely, while looking for enemy troops to shoot. Units can also be assigned to workstations like turrets, to have them control the turret.

Setting trench excavation paths is done by pressing CTRL and dragging your mouse along the path, scrolling the mouse wheel to determine the depth of the trench, then pressing ENTER to confirm it. Once a path is set enemies in excavation mode will be able to fulfill their objective by digging out the trenches. If the excavation path crosses through a patch of resources (i.e. compact organic material or energy crystals) the troops will dig up those materials and depot them at the nearest item depot. Alternatively, troops may deliver resources to a nearby building, if said building has requested those materials (either for its own construction, or for production reasons - like how the Foot Unit Factory needs compact organic material to produce Foot Units).

Player Feedback

Each unit has an inventory that can be viewed when the given unit is selected. Multiple enemies can be selected at once by clicking and dragging the mouse, which shows a semi-transparent bounding box over the selected area. All units within the box will be selected once the mouse button stops being pressed.

Units' current states and actions are shown using an icon above their head. Here's what the icons represent:

- Shovel icon: Unit is in excavating mode
- Walking icon: Unit is in patrol mode
- Crate icon: Unit is in transport mode
- Clock icon: Unit is in an idle state
- Target icon: Unit is attacking
- Orange arrow icon: Unit is walking safely
- Red arrow icon: Unit is walking directly

Player feedback is also given when selecting trench excavation paths. When a player is selecting an excavation path a line of arrows will appear on the selected areas. The arrows first appear blue and despawn after a unit has excavated the area.

Selecting a Foot Unit will show its current target destination as a ring on the ground. If you select one of the “Move” commands and choose a location, the ring will move there to denote that position as the Foot Unit’s new target destination. A similar thing happens with the Mortar Unit, except instead it’s the mortar’s firing target. When the player selects the “Set Target” option the ring will start following the cursor (and the mortar will stop firing) until the player clicks the new target position.

Weapons also have a visible ammo shot when firing. Foot unit guns shoot a regular bullet line towards their target, as do turrets. Mortars on the other hand shoot out a rocket into the sky surrounded by particles, which explodes on impact leaving a mark in the ground.

Sound effects are also played when units are shooting, which help alert the player of danger from the enemy team, the sound is managed in 3D so the sound gets louder the closer the player camera gets to the cause of the sound. There are different sounds for turrets and mortars.

Additional Features

Dynamic Editable Terrain
The core mechanic of our game is the dynamic editable terrain system, which allows for the terrain heightmap to be modified at runtime. This mechanic is essential to enabling the trench warfare dynamics we desire. The dynamic editable terrain is implemented via a compute shader that operates on the heightmap that determines the vertex displacement of each terrain tile. Because the terrain heightmap data is also needed on the CPU to facilitate collision detection, a GPU readback queueing system was implemented to cap the maximum amount of data being read back each frame, allowing for smooth performance even when many edits are taking place at once. Because parts of the terrain are composed of minable resources, we must also be able to determine how much of each resource was extracted in a given terrain mining operation. This is also handled in the compute shader, making use of the ImageAtomicAdd function to increment the values in a texture corresponding to the mined resources of the unit doing the mining. For example, digging within a patch of the red material will award the troop some Compacted Organic Material, with the amount corresponding to how much was dug up.

Networked Multiplayer
Because the primary mode of play we had in mind when developing this game was player vs player, we implemented networked multiplayer. Games may be hosted and joined over a local-area-network connection.

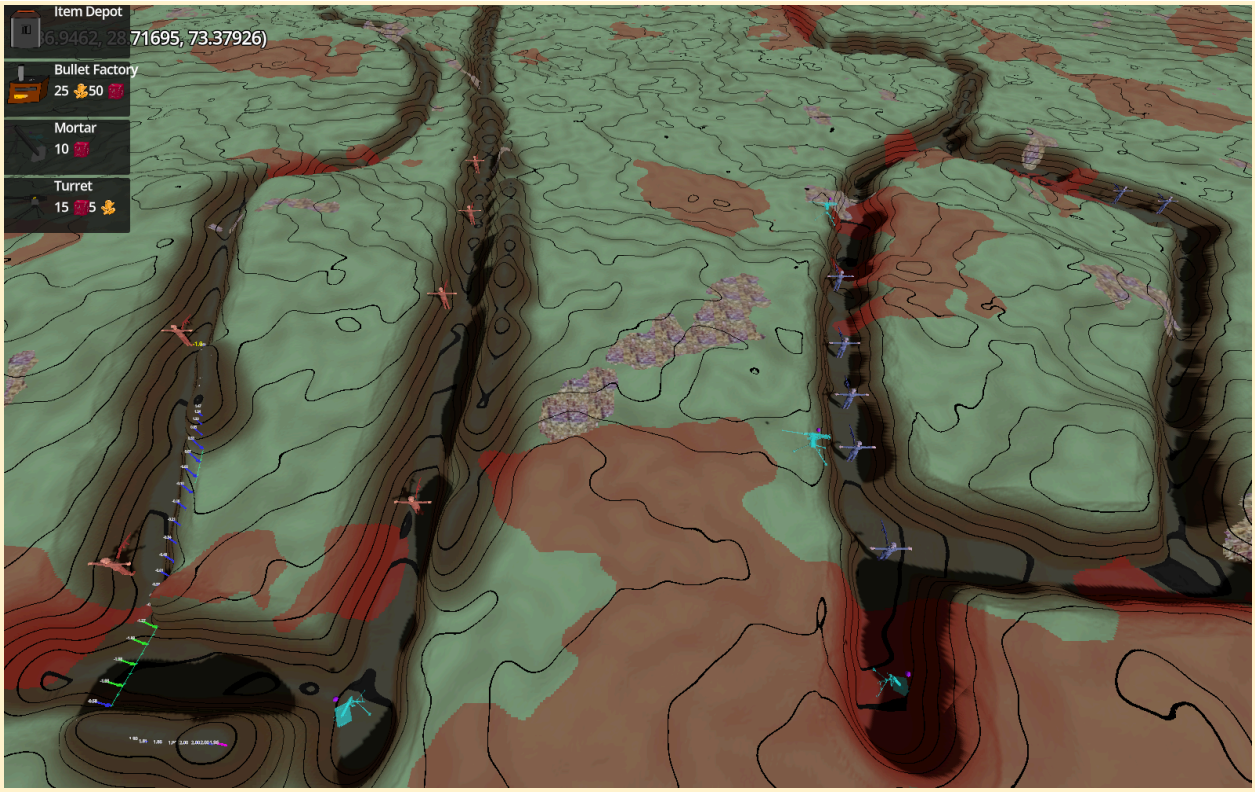
Unit Visibility-Based Fog of War
The visibility of enemy units depends on one or more of your own units having line-of-sight to them. In order to maintain performance with large numbers of units, checks are amortized over

multiple frames and a set of caching techniques are employed. From performance testing, line-of-sight checking in 128v128 unit scenarios performs extremely well.

Another additional feature is music and sound effects, sound effects are played with various actions in the game like shooting and music is played consistently throughout the game. The music was composed by team member Hendry Hu.

There is also a robust title screen that includes toggling the shadows for performance and adjusting the volumes for the music and sound effects separately.

Hall of Fame Image:



Video Demo: https://youtu.be/2YXFHon6_Q4